How synth settings are stored

QB Nebulae V2: the Csound, SuperCollider and Pure Data banks. Branch `refactor_controlhandler`, commit bc95c6f, 2026-10-17.

Short version: there is **one shared settings file for all three banks**, and it is only written when you change mode. Only the Csound bank actually updates its values; SuperCollider and Pure Data never put new values into it.

1. What is stored, and where

File	Contents	Written by	Read by
`Code/config/bootinstr.txt`	`bank,<instr|supercollider|puredata>` and `instr,<name>`	`Nebulae.writeBootInstr()` (nebulae.py:214)	`Nebulae.__init__` (nebulae.py:47)
`Code/config/nebsettings.txt`	`name,value` for every control: speed, pitch, ..., file, plus all *_alt ones	`SettingManager.write()` (settings.py:49)	`SettingManager.read()` (settings.py:20)
`Code/config/defaultnebsettings.txt`	Factory defaults, same format	never (read-only)	`SettingManager.read()`
`nebconfigbegin ... nebconfigend` block inside the .instr / .scd file	Settings for each patch (ksmps, sr, button modes, *_alt defaults)	the patch author	`InstrParser.parse()` (instrparser.py:10)

There is **no file per instrument or per bank**. Every Csound instrument, SC patch and PD patch shares the same `nebsettings.txt`.

2. The classes involved

SettingManager (settings.py)

Holds three dicts:

- `defaultDict`: built from `defaultnebsettings.txt`. Any *_alt key found in the patch's nebconfig block overrides the file value (`populateDefaultDict`, line 125).
- `settingDict`: built from `nebsettings.txt`, or from the defaults if that file is missing or empty.
- `mutableSettings`: a working copy that changes at runtime.

Its methods:

- `load(key)`: reads from `settingDict`. Used once, as the starting value of each `ControlChannel`.
- `save(key, val)`: only writes to `mutableSettings` in RAM, nothing goes to disk.
- `update(now)`: copies `mutableSettings` into `settingDict` and builds the lines to write.
- `write()`: remounts the filesystem read-write, writes the file, then remounts read-only.
- `offloadWrite()` is never used (it's marked TODO).

ControlHandler (controlhandler.py)

Creates a `SettingManager` and calls `read()` in `__init__` (lines 72-73). It then passes `settings.load(name)` as the starting value of every channel (lines 115-143). Because `SCControlHandler` and `PDControlHandler` inherit this, a **new SettingManager and a fresh read happen every time a handler is created**, which means on every bank or patch load.

InstrParser (instrparser.py), via Conductor.getConfigDict()

Supplies the `configData` that overrides the *_alt defaults.

3. When each step happens

Boot

`Nebulae.__init__` reads `bootinstr.txt` and then calls `start()`, `start_supercollider()` or `start_puredata()`. Each of these creates a control handler, which reads `nebsettings.txt`.

Csound bank (ControlHandler)

- Every loop in `updateAll()`, it calls `settings.save()` for each channel. Encoder channels (speed/pitch) save `getStaticVal()`, other channels save `getValue()` (lines 517-523). In secondary mode it saves the alt channels (line 478). All of this stays in RAM.
- It only writes to disk in `enterNormalMode()`, `enterSecondaryMode()` and `enterInstrSelMode()` (lines 272, 289, 298), each of which calls `update()` and then `write()`.
- **Anything changed since the last mode change is lost on power-off.**

SuperCollider bank (SCControlHandler)

- `updateAll()` only sends OSC messages and **never calls** `settings.save()`.
- `enterSuperColliderMode()` and the inherited `enterSecondaryMode()` do call `update()` and `write()`. But since `mutableSettings` never changed, they just write back the values that were loaded at startup.

Pure Data bank (PDControlHandler)

- Same as SC: no `save()` in `updateAll()`.
- `enterPureDataMode()` doesn't even call `update()` or `write()`; only the inherited `enterSecondaryMode()` and `enterInstrSelMode()` write.
- It is created with `configData=None` (nebulae.py:359), so the patch's *_alt overrides never apply.

Changing patch

- `run()`, `run_supercollider()` and `run_puredata()` call `writeBootInstr()` and then start the new bank.
- `restoreAltToDefault()` sets every alt channel back to `getDefault()`. It is called:
  - by `start()` when the instrument changed (nebulae.py:153),
  - by `run()` on every run after the first (nebulae.py:162),
  - by the UI on a pitch click in secondary mode (ui.py:736).
- Note that it only sets the channel values; it doesn't call `save()`.

Summary by bank

	Csound	SuperCollider	Pure Data
Handler class	`ControlHandler`	`SCControlHandler`	`PDControlHandler`
Settings read	on handler creation	on handler creation	on handler creation
save() in updateAll()	yes, every loop	no	no
Written to disk	enterNormal / Secondary / InstrSel	enterSuperCollider / Secondary / InstrSel (unchanged values)	enterSecondary / InstrSel (unchanged values)
Patch *_alt defaults	from .instr nebconfig	from .scd nebconfig	not applied (configData=None)

4. What this means in practice

- In SC and PD, knob, encoder and alt values are **never persisted**. The file keeps whatever the last Csound session wrote, and SC/PD start from those Csound values.
- The settings are global, not per patch: speed, pitch and alt values carry over between unrelated instruments. Only the alt values get reset to defaults when you switch.
- For the analog pots, the saved value hardly matters, because the physical pot position replaces it on the first read. What persistence really affects is the encoders (speed/pitch) and the alt/secondary values.
- Disk writes are tied to mode changes, not to value changes or shutdown.

Suggestion for the refactor: if SC and PD should get their own settings handling, the obvious place is the `updateAll()` of each subclass: add the `settings.save()` calls there, and consider a separate settings file per bank or per patch.